2026|04 AKIA3

Pearson Korrelation:

Kernbegriffe & Formeln (Kurzübersicht)

r - Pearson-Korrelationskoeffizient:
Misst die stärke und richtung des lineare Zusammenhänge zwischen zwei Variabeln

$r \in [-1, 1]$

KORRELATIONSMATRIX

$$\rho_{xy} = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2 \sum_{i=1}^{n} (y_i - \bar{y})^2}$$

Pandas: df.corr() | Numpy np.corrcoeff(x, y)[0, 1]

t - t-statistik (Prüfgröße)

Reines r reicht nicht! t wandelt r unter Berücksichtigung der stichprobengröße n in eine Signal-zu-Rausch Verhältnis um.

df (Vorsicht: kein Pandas Dataframe!) - Degrees of Freedom

Zahl der mathematisch frei variierbaren Datenpunkte nach Abzug geschätzter Parameter.

Example: $df = n - 2$

p (p-wert) (Signifikanz): Wahrscheinlichkeit, ein solches r rein durch Zufall zu beobachten, wenn in Wahrheit $\rho = 0$ gilt.

⇒ Der Null Hypothese H_0 ist: x und y nicht korreliert sind.

⇒ Wir können H_0 ablehnen wenn p niedrig ist!

Englisch: "If p is low, the null must go!"

Normalerweis $p < 0.05$

Test mit Toy Dataset von HH's Lecture

```python
# Toy Datenstz aus en Vorlesungsnotizen

x_toy = np.array([15, 12, 9])
y_toy = np.array([1500, 1800, 2100])

# Run Pearson Test on toy dataset
r_toy, t_toy, df_toy, p_toy = pearson_test(x_toy, y_toy)
print(f"[Toy Data] r = {r_toy:.4f}, t = {t_toy:.4f}, df = {df_toy}, p = {p_toy:.4f}")

# Attempt Fisher CI on toy dataset
try:
    ci_lo_toy, ci_hi_toy = fisher_ci(r_toy, n=len(x_toy))
    print(f"[Toy Data] 95%-CI = ({ci_lo_toy:.4f}, {ci_hi_toy:.4f})")
except Exception as e:
    print(f"[Toy Data] Fisher CI calculation failed: {e}")


# Correlation Matrix on Toy dataset
df_toy = pd.DataFrame({"x_minuten": x_toy, "y_schritte": y_toy})
R_toy = df_toy.corr(method="pearson")
print(f"[Toy Data] Correlation Matrix:\n{R_toy}")
```

✓ 0.0s Python

```
[Toy Data] r = -1.0000, t = -1000000.0000, df = 1, p = 0.0000
[Toy Data] 95%-CI = (-1.0000, 1.0000)
[Toy Data] Correlation Matrix:
           x_minuten  y_schritte
x_minuten        1.0        -1.0
y_schritte      -1.0         1.0
/tmp/ipykernel_6312/2578799129.py:33: RuntimeWarning: divide by zero encountered in scalar divide
  se = 1.0 / np.sqrt(n - 3)
```

Müssen wir die Null Hypothese H_0 verwerfen (ablehnen)?

⇒ Rein rechnerisch (Formel): Ja, da $p < 0.05$ ist

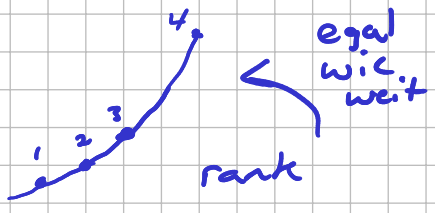
⇒ In der Praxis: Nein! das Ergebnis ist unbrauchbar!

⇒ Bei $n=3$, haben wir $df=1$

Nicht-Lineare Korrelationen

Maßzahl	Funktionsweise	Wann nutzen?
Spearman	Rangkorrelation: Wendet die Pearson-Formel auf die Ränge $d_i = \text{rank}(x_i) - \text{rank}(y_i)$ statt Rohwerte an. ⇒ Prüft auf monotone Zusammenhänge	Bei gekrümmten, aber streng steigend/fallend Trends (z.B. exponentiell) oder starken Ausreißern.
Kendall's Tau $\tau = \frac{(C-D)}{\frac{n(n-1)}{2}}$	Paarweise Konkordanz C = Konkordante Paare (gleiche Richtung) D = Diskordante Paare (gegensätzige Richtung)	Bei kleinen Stichproben ($n < 30$)
Distance Korrelation dCor	Distanzbasierte Abhängigkeit Nutzt doppelt zentrierte Distanzmatrizen Es gilt: $dCor = 0 \Leftrightarrow$ X und Y sind komplett unabhängig!	Erkennt alle strukturellen Abhängigkeiten Z.B. U-Formen, Parabeln, Wellen, usw.

Monotone ⇒ Strictly increasing
Strictly decreasing



Automatisiertes Screening Nach Nicht-Linearen Paaren

⇒ Das Suchkriterium: Großes dCor und kleines |r|

⇒ In großen Datensätzen ist das manuell Betrachten aller Scatterplots unmöglich. Um versteckte nicht-Lineare Muster zu finden, suchen wir gezielt mit Variablenpaaren mit:

Großem dCor (>0.3): zeigt, dass eine starke allgem. Abhängigkeit zwischen die Variabeln existiert

kleinem |r| (r=0.0): zeigt, dass keine lineare Beziehung vorliegt

Die mathematische Kennzahl für unser automatisiertes Screening ist die nicht-lineare Lücke:

Nonlinear_Gap = dCor - |r|

Ziel: Maximierung diese Lücke!

```python
import itertools

def find_nonlinear_pairs(df):
    results = []
    cols = df.select_dtypes(include=[np.number]).columns

    # Schleife über alle eindeutigen Variablenpaare
    for col1, col2 in itertools.combinations(cols, 2):
        x = df[col1].dropna()
        y = df[col2].dropna()

        common_idx = x.index.intersection(y.index)
        if len(common_idx) < 10:
            continue  # Nicht genügend Datenpunkte

        x_vals, y_vals = x.loc[common_idx], y.loc[common_idx]

        # Pearson r und Distance Correlation berechnen
        r = np.corrcoef(x_vals, y_vals)[0, 1]
        dcor = distance_correlation(x_vals, y_vals)

        # Nicht-Lineare Lücke: hihes dCor bei geringer linearer Korrelation
        gap = dcor - abs(r)

        results.append({
            "Pair": f"{col1} Vs {col2}",
            "Pearson r": round(r, 3),
            "Distance Correlation": round(dcor, 3),
            "Nonlinear Gap": round(gap, 3)
        })

    return pd.DataFrame(results).sort_values(by="Nonlinear Gap", ascending=False)
```

0.0s

You have Docker installed on your system. Do you want to

```python
# Einlesen Daten (CSV) und Suche nach nichtlinearen Beziehungen (Student Productivity Dataset)
df = pd.read_csv("archive/student_productivity_distraction_dataset_20000.csv", nrows=1000)  # Beispiel: nur die ersten 1000 Zeilen einlesen

# Reinschauen
print(df.head())
```

0.0s Python

```
   student_id  age  gender  study_hours_per_day  sleep_hours  \
0           1   23  Female                 4.35         3.63
1           2   20    Male                 6.14         6.58
2           3   29  Female                 4.98         3.26
3           4   27  Female                 3.19         4.58
4           5   24    Male                 7.67         6.21

   phone_usage_hours  social_media_hours  youtube_hours  gaming_hours  \
0               3.38                2.73           1.83          5.26
1               5.48                1.51           3.13          1.73
2               4.83                3.63           0.18          4.71
3              10.06                3.95           5.75          2.52
4               3.02                1.59           5.46          5.65

   breaks_per_day  coffee_intake_mg  exercise_minutes  assignments_completed  \
0               6               347               111                      2
1              13               403                28                     10
2               1               419               102                      8
3               9               178                28                     18
4               8               436               105                      7

   attendance_percentage  stress_level  focus_score  final_grade  \
```

```python
# Nicht-numerische Spalten entfernen
df_clean = df.drop(columns=["student_id", "gender"], errors='ignore')  # Fehler ignorieren, falls Spalten nicht existieren

# Alle vrbleibenden Spalten explizit auf float konvertieren
df_clean = df_clean.astype(float)

screening_result = find_nonlinear_pairs(df_clean)
print(screening_result.head(10))  # Zeige die Top 10 Paare mit der größten nichtlinearen Lücke
```

✓ 2.4s Python

```
                                        Pair  Pearson r  \
112          attendance_percentage Vs final_grade      0.001
26             study_hours_per_day Vs focus_score      0.005
64       social_media_hours Vs productivity_score      0.002
96               coffee_intake_mg Vs focus_score      0.006
101             exercise_minutes Vs stress_level      0.003
94      coffee_intake_mg Vs attendance_percentage     -0.005
109   assignments_completed Vs productivity_score      0.001
100     exercise_minutes Vs attendance_percentage     -0.001
30              sleep_hours Vs social_media_hours     -0.008
23   study_hours_per_day Vs assignments_completed      0.002

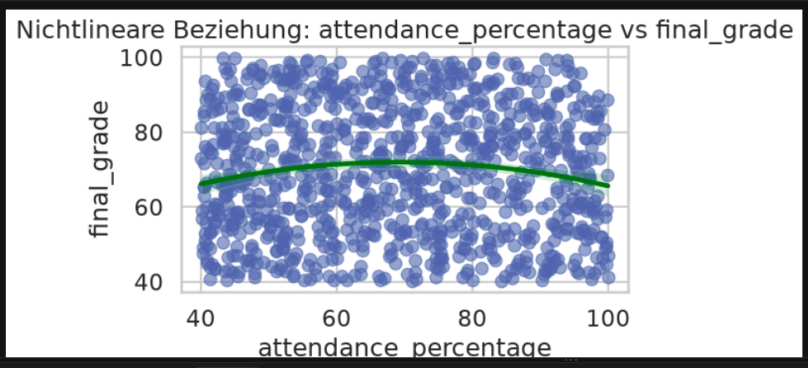
     Distance Correlation  Nonlinear Gap
112                 0.059          0.058
26                  0.053          0.048
64                  0.049          0.046
96                  0.050          0.044
101                 0.046          0.043
94                  0.048          0.043
109                 0.043          0.042
100                 0.043          0.042
30                  0.049          0.041
23                  0.043          0.041
```

```python
# Dynmische Auslesen des Variabelnpaars mit dem höchsten Nonlinear_Gap
top_pair = screening_result.iloc[0]["Pair"]
top_x, top_y = top_pair.split(" Vs ")

plt.figure(figsize=(6,4))
sns.regplot(x=top_x, y=top_y, data=df_clean, order=2, scatter_kws={"alpha":0.6}, line_kws={"color":"darkgreen"})

plt.title(f"Nichtlineare Beziehung: {top_x} vs {top_y}")
plt.xlabel(top_x)
plt.ylabel(top_y)
plt.tight_layout()
plt.show()
```

✓ 0.2s Python



Versteckte Nicht-lineare Beziehung!

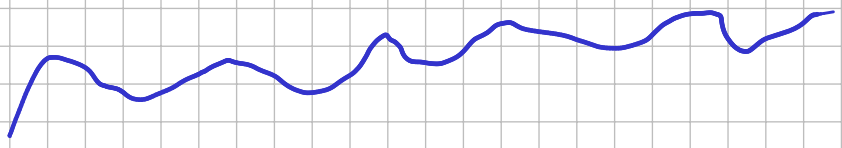
ARIMA

Stationäritäät

AR (Autoregressiv)

I (Integriert -> "Differencing") $\Delta y = y_t - y_{t-1}$

MA (Moving Average - Gleitende Durchschnitt)



Beispiel:

```python
# --- EXERCISE: The Importance of Differencing (d=0 vs d=1) ---

# 1. Fit ARIMA without differencing (d=0)
model_d0 = ARIMA(y_train, order=(1, 0, 1)).fit()
fc_d0 = model_d0.get_forecast(steps=len(y_test)).predicted_mean
fc_d0.index = y_test.index

# 2. Fit ARIMA with differencing (d=1)
model_d1 = ARIMA(y_train, order=(1, 1, 1)).fit()
fc_d1 = model_d1.get_forecast(steps=len(y_test)).predicted_mean
fc_d1.index = y_test.index

# 3. Compare Forecasts against actual Test Data
plt.figure(figsize=(12, 5))
y_train.tail(90).plot(label="Train (Last 90 days)", color="tab:blue")
y_test.plot(label="Actual Test Data", color="black", linewidth=2)
fc_d0.plot(label="d=0: No Differencing (Broken)", color="crimson", linestyle="--")
fc_d1.plot(label="d=1: With Differencing (Captures Trend)", color="green")

plt.title("Why Differencing Matters: Predicting Non-Stationary Data")
plt.legend()
plt.tight_layout()
plt.show()

# 4. Compare Error Metrics
print("d=0 (No Diff) MAE:", mean_absolute_error(y_test, fc_d0))
print("d=1 (Diff)    MAE:", mean_absolute_error(y_test, fc_d1))
```

✓ 0.4s Python

```
/home/chuck/Dropbox/Thesis/CGH4JOM/RP1/Code/.venv/lib/python3.12/site-packages/statsmodels/tsa/statespace/mlemodel.py:679: EstimationWarning:
```

